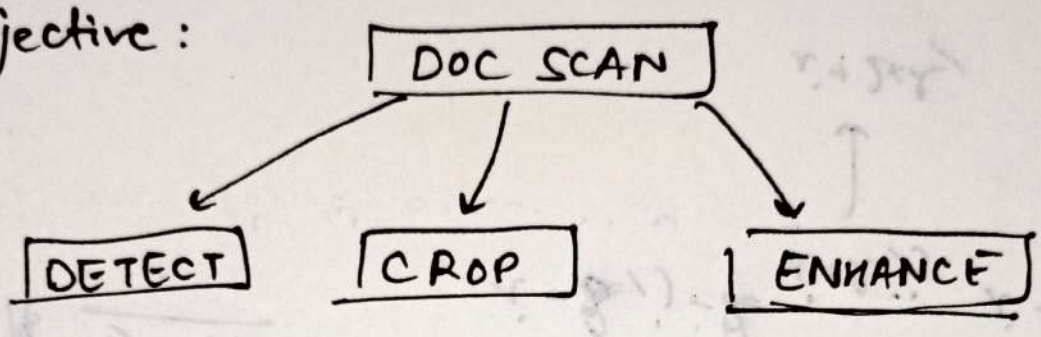
DOC - SCANNER.

Repo: gh- doc-scanner

Objective:



Operations : Image operations

Terms:

1) Document -driven development.
(To define a project purpose before writing a single code).

2) Licensing / Legal compliance:
Process of ensuring that software we build data we handle, tools we use should adhere to laws and regulation of jurisdictions where the software operates.

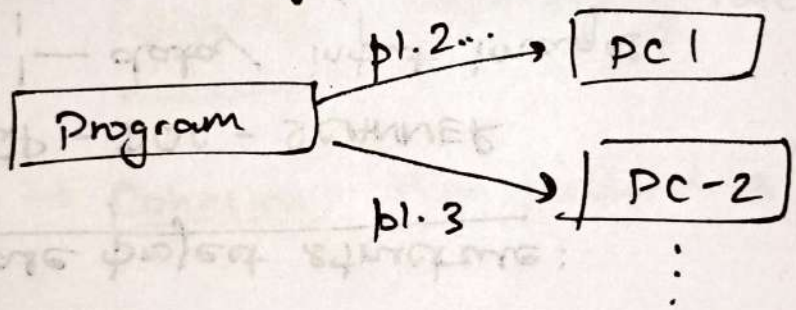
A non-functional requirement.

→ MIT Licence: Do whatever you want with my code, just don't sue me if it breaks, keep my name on the original license.

3) Repo. hygiene: Using gitignore, it tells GitHub to never upload junk files, keeping the codebase professional and lightweight.

- - - - -

Project manager: (uv)



To avoid the conflict of seperate packages for a single platform.

venv: Virtual enviornment: A private sandbox for the project, uv creates a local folder where all dependencies (project specific) live.

Dependency resolution: Process to find out which version of different libraries can work together without crashing.

uv.lock

Lockfile: Records exact version of single library (and sub-libraries)

Prevents: it works only on my machine bug.

pyproject.toml.

Project metadata: ID card of the project.

name, version, list of every library needed.

PEP 621 format

Base project structure:

```
GP - DOC - SCANNER
 |— data/ input images
 |
 |— output/ Final PDFs.
 |
 |— src/ SOURCE code
 |    |— __init__.py → src as a package
 |    |— main.py → Manager code
 |    |— scanner.py → Worker code
 |    |— utils.py → Helper code
 |
 |— tests/ → Unit testing.
```

Terms:

1) Seperation of Concerns: A computer program must divide into distinct section where each section addresses a certain responsibility or "concern".

→ Modularization: Breaking a big project into smaller, "independent" modules/ files.

→ Encapsulation: Hiding the how and showing the what?

→ Coupling: Dependency rate between two modules

→ Cohesion: How focused a single module is.

GitHub commands: Local to remote code sync.

1. git add . → Staging: Put a ☑ on every file you want to include in the next delievery.

⇒ Change 10 files but save only 5, git add helps to pick what exactly to ~~change~~ go to the save point

2. git commit -m "Message":

Atomic History: Take a snapshot of the work area and add a label.

chore : Setup

feat : new feature

fix : for bugs.

3. git push origin main : <u>Remote synchronization</u>

⇒ Take that labeled snapshot and upload to GitHub cloud.